

Lambdas, Action<T>, and Func<T,TResult>



Dan Wahlin

Wahlin Consulting

@DanWahlin | www.codewithdan.com



Overview



Understanding Lambdas and Delegates

Using `Action<T>`

Using `Func<T, TResult>`

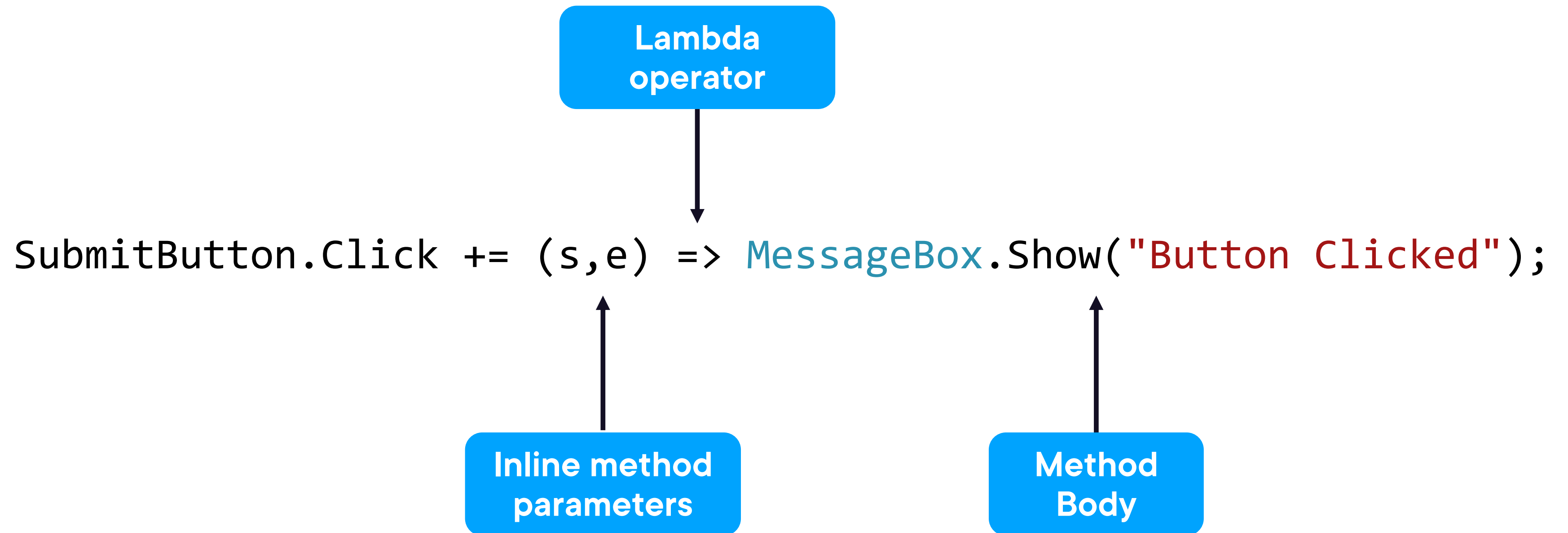




Understanding Lambdas and Delegates



Understanding Lambda Expressions



```
delegate int AddDelegate(int a, int b);

static void Main(string[] args)
{
    AddDelegate ad = (a,b) => a + b;
    int result = ad(1,1); //result = 2
}
```

Assigning a Lambda to a Delegate

Lambda expressions can be assigned to any delegate.



```
delegate bool LogDelegate();

static void Main(string[] args)
{
    LogDelegate ld = () =>
    {
        WriteToEventLog();
        return true;
    };
    bool status = ld();
}
```

Handling Empty Parameters

Delegates that don't accept any parameters can be handled using lambdas.





Lambdas in Action

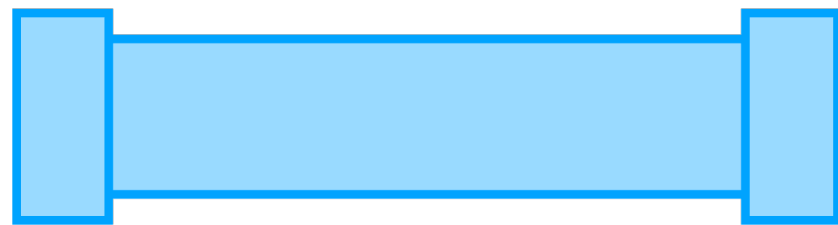




Using Action<T>



Delegates in .NET



The .NET framework provides several different delegates that provide flexible options:

- Action<T> - Accepts one or more parameters and returns no value
- Func<T,TResult> - Accepts one or more parameters and returns a value of type TResult



```
public static void Main(string[] args)
{
    Action<string> messageTarget;
    if (args.Length > 1) messageTarget = ShowWindowsMessage;
    else messageTarget = Console.WriteLine;
    messageTarget("Invoking Action!");
}

private static void ShowWindowsMessage(string message)
{
    MessageBox.Show(message);
}
```

Using Action<T>

Action<T> can be used to call a method that accepts a single parameter of type T.



Using `Func<T,TResult>`



```
public static void Main(string[] args)
{
    Func<string, bool> logFunc;
    if (args[0] == "EventLog") logFunc = LogToEventLog;
    else logFunc = LogToFile;
    bool status = logFunc("Log Message");
}

private static bool LogToEventLog(string message) { /* log */ }

private static bool LogToFile(string message) { /*log */ }
```

Using Func<T,TResult>

Func<T,TResult> supports one or more parameters (T) and returns a value (TResult).



Summary



Lambdas provide a way to define inline methods using a concise syntax

.NET provides built-in delegate types such as:

- Action<T>
- Func<T, TResult>



Event Handler

EventArgs

Event Raiser

Delegate



Thank You!



Dan Wahlin
Wahlin Consulting

@DanWahlin
<https://codewithdan.com>

